

Nachrichten unter Beobachtung: Eine wissenschaftliche Arbeit zu News-Hash

Ein neuer Forschungsbericht über lokale Überlieferung, technische Provenienz und prüfbare Zeitbezüge digitaler Nachrichten

News-Hash-Projekt · Wissenschaftliche Arbeit · 17. August 2026

Transparenzhinweis: Die News-Hash-App und diese Arbeit wurden fast vollständig mit KI erzeugt. Menschen legten Ziel, Auswahl und Prüfmaßstäbe fest und verantworten die Freigabe.

„Mit KI gebaut, mit Liebe verfeinert, für neugierige Menschen.“

Abstract. Digitale Nachrichten besitzen eine hohe gesellschaftliche Relevanz, aber eine geringe materielle Stabilität. Webtexte ändern sich, Feeds liefern oft nur Ausschnitte und verlinkte Ressourcen können verschwinden. Diese Arbeit untersucht News-Hash als lokalen Forschungsprototyp für eine nachvollziehbare Nachrichtenüberlieferung. Im Zentrum stehen ein quellenbezogener Importprozess, eine deterministische

Normalisierung, parallele Speicher in JSONL und SQLite, verkettete Hashwerte sowie ein öffentliches Zeitmanifest. Die Arbeit entwickelt ein Modell, in dem Inhalt, Verarbeitungsversion und Speicherzustand gemeinsam geprüft werden können. Sie diskutiert außerdem Screenshots als Beobachtungsform, OpenTimestamps als externen Zeitbezug, Fehler- und Bedrohungsszenarien, Datenschutz und die Rolle generativer KI bei der Softwareentstehung. Das Ergebnis ist kein Wahrheitsautomat, sondern ein begrenztes Verfahren zur technischen Rekonstruktion dessen, was ein lokaler Archivprozess zu einem bestimmten Zeitpunkt gespeichert hat.

Schlüsselwörter: Nachrichtenforschung, Webarchivierung, Provenienz, Datenintegrität, Hash-Kette, OpenTimestamps, KI-Transparenz

1. Ausgangspunkt: Nachrichten als flüchtige Quellen

Nachrichten werden heute überwiegend als veränderliche Webressourcen wahrgenommen. Eine Adresse verweist nicht zwingend auf eine historische Fassung, sondern meist auf den

gegenwärtig ausgespielten Zustand eines Dokuments. Überschriften werden angepasst, Artikel gekürzt, Bilder ersetzt und Beiträge entfernt. Für eine spätere Rekonstruktion reicht deshalb ein Link nicht aus. Es muss zusätzlich nachvollziehbar sein, welche Repräsentation beobachtet und aufbewahrt wurde.

Die Flüchtigkeit ist nicht nur ein technisches Problem. Sie beeinflusst journalistische Nachprüfung, wissenschaftliche Quellenarbeit und die Erinnerung gesellschaftlicher Ereignisse. Wer einen früheren Text analysiert, benötigt Angaben zu Abrufzeit, Quelle, Inhalt und möglicher Veränderung. Ein Archiv, das nur aktuelle Fassungen sammelt, kann die historische Entwicklung nicht zuverlässig zeigen. Ein Archiv, das jede Kopie unmarkiert ablegt, kann dagegen die eigene Herkunft nicht erklären.

News-Hash setzt an dieser Spannung an. Das Projekt wählt keinen Anspruch auf vollständige Webkopie, sondern dokumentiert einen abgegrenzten Beobachtungsvorgang. Ein Feed wird abgerufen, ein Datensatz entsteht, ein Hash wird

berechnet und der Datensatz wird mit der vorherigen Beobachtung verbunden. Die Methode verschiebt den Schwerpunkt von der bloßen Verfügbarkeit zur überprüfbaren Entstehung einer lokalen Archivspur.

2. Forschungsproblem und Erkenntnisinteresse

Das Forschungsproblem lautet: Wie kann ein kleiner lokaler Dienst digitale Nachrichten so erfassen, dass spätere Änderungen an der gespeicherten Fassung und an ihrer zeitlichen Einordnung erkennbar werden? Die Frage umfasst mehrere Ebenen. Sie betrifft die Identität einer Meldung, die Transformation heterogener Feeddaten, die Persistenz in unterschiedlichen Formaten und den Umgang mit einem externen Zeitnachweis.

Das Erkenntnisinteresse richtet sich nicht auf eine neue Hashfunktion. Es gilt der Verbindung etablierter Verfahren zu einem Arbeitsablauf, dessen Annahmen offenliegen. Ein wissenschaftlich brauchbarer Prototyp muss nicht jede mögliche Quelle abdecken. Er muss aber zeigen, welche

Eigenschaften er bewahrt, welche er nur schätzt und an welcher Stelle die Aussage endet.

Aus dem Problem ergeben sich die Fragen, wie Quelleninhalte formalisiert werden, wie Codecs unterschiedliche Beobachtungen erzeugen, wie zwei Speicherketten unabhängig bleiben und wie ein Manifest geprüft werden kann. Ergänzend wird untersucht, ob eine explizite Versionierung von Schema, Codec und Hashdefinition die Reproduzierbarkeit älterer Records verbessert.

3. Forschungsfragen

Die erste Forschungsfrage betrifft die Repräsentation: Welche Felder müssen in einem Nachrichtenrecord enthalten sein, damit ein späterer Prüfer Inhalt, Quelle und Zeitbezug unterscheiden kann? Die zweite betrifft die Integrität: Welche Kettenregel macht eine Veränderung oder Umordnung sichtbar, ohne eine globale Transaktion über alle Speichermedien zu benötigen?

Die dritte Frage untersucht Erweiterbarkeit. Feeds liefern häufig lediglich Anreißer. Wie kann ein System vollständige Artikel oder visuelle Zustände nachladen, ohne die allgemeine Importlogik mit Einzelfällen zu überladen? Die vierte Frage richtet sich auf Belege: Wie lassen sich Manifest, OTS-Proof, SQLite-Artefakt und Shardnummer so verbinden, dass ein konkreter Hash auffindbar bleibt?

Die fünfte Frage betrifft Verantwortung. Welche technischen und organisatorischen Grenzen müssen dokumentiert werden, damit ein unveränderlicher Record nicht fälschlich als Beweis journalistischer Wahrheit verstanden wird? Diese Frage umfasst Datenschutz, Urheberrecht, Auswahl der Quellen und den hohen Anteil generativer KI an App und Arbeit.

4. Theoretischer Bezugsrahmen

Für digitale Langzeitarchivierung ist das OAIS-Referenzmodell ein wichtiger Bezugsrahmen [6]. Es unterscheidet unter anderem Übernahme, Verwaltung, Erhaltung und Zugriff. News-Hash bildet diese Ebenen nicht

vollständig nach, übernimmt aber die grundlegende Einsicht, dass Archivierung mehr ist als das Kopieren einer Datei. Eine Übernahme erzeugt ein Informationsobjekt, dessen Kontext und Erhaltungsbedingungen ebenfalls beschrieben werden müssen.

Das W3C-PROV-Modell versteht Provenienz als Information über Entitäten, Aktivitäten und beteiligte Akteure [7]. Für News-Hash bedeutet dies: Die Nachricht ist eine Entität, der Feedabruf und die Codecverarbeitung sind Aktivitäten, und der lokale Betreiber beziehungsweise die konfigurierte Quelle gehören zum Herkunftskontext. Die bestehende Anwendung speichert noch keinen vollständigen PROV-Graphen, folgt aber einer ähnlichen Trennung von Gegenstand und Verarbeitung.

Das Memento-Framework beschreibt den Zugriff auf frühere Zustände von Webressourcen über Zeitverhandlung und TimeMaps [8]. News-Hash verfolgt einen anderen Schwerpunkt. Es stellt keine allgemeine Zeitverhandlung für fremde HTTP-Ressourcen bereit, sondern erzeugt lokale

Snapshots und versieht ihre Sequenz mit Hashbeziehungen. Die beiden Ansätze sind komplementär: Memento adressiert Auffindbarkeit vergangener Webzustände, News-Hash die Integrität eines lokal beobachteten Zustands.

5. Forschungslogik und Grenzen

Die Arbeit ist konstruktionsorientiert. Aus den Forschungsfragen werden technische Invarianten und Tests abgeleitet. Ein Record darf im normalen Betrieb nicht überschrieben werden. Ein Folgehash muss seinen Vorgänger referenzieren. Ein v1-Record muss die Definitionen seiner Verarbeitung über Hashwerte identifizieren. Ein Manifest muss denjenigen Shard nennen, in dem der Endhash zu finden ist. Diese Invarianten bilden die Brücke zwischen Theorie und Implementierung.

Die Arbeit ist keine Untersuchung von Medienqualität und keine repräsentative Analyse des Nachrichtenmarkts. Die Quellenkonfiguration ist ein konkreter Ausschnitt. Die Tests beschreiben erwartete Systempfade, keine allgemeine

Eigenschaft des gesamten Webs. Aussagen über journalistische Wahrheit, Vollständigkeit oder gesellschaftliche Neutralität liegen außerhalb der technischen Evidenz.

Der hohe KI-Anteil wird als methodische Randbedingung dokumentiert. Generative Werkzeuge können Code, Gliederungen und Formulierungen erzeugen, aber sie prüfen keine externe Wirklichkeit automatisch. Daher werden technische Aussagen mit Quellcode, Tests und Primärdokumenten abgeglichen. Die menschliche Leistung liegt in Zielsetzung, Auswahl, Korrektur und Verantwortung.

6. Quellenraum und Beobachtungseinheit

Die Beobachtungseinheit ist nicht „die Nachricht an sich“, sondern die Darstellung, die ein konfigurierter Abrufprozess zu einem Zeitpunkt erhält. Ein RSS-Feed kann Titel, Link, Kurztext und Datum liefern. Eine nachgeladene Seite kann zusätzlichen Artikeltext, Bilder oder dynamische Elemente enthalten. Der Record muss daher kenntlich machen, welcher

Codec verwendet wurde und welche Form der Beobachtung vorliegt.

Quellen werden in TOML konfiguriert. Name, Feed-URL, Speichername, Codec und Pollingintervall bestimmen den Rahmen. Die Konfiguration ist Teil der Forschungsumgebung. Eine Änderung der Quellenliste verändert, welche Welt das Archiv sichtbar macht. Deshalb sollten Konfigurationsänderungen versioniert und gemeinsam mit den Prüfergebnissen dokumentiert werden.

Die Auswahl ist notwendig, aber nicht neutral. Ein Feed mit hoher technischer Verfügbarkeit wird leichter archiviert als eine Paywall, eine dynamische Anwendung oder eine Quelle ohne RSS. Die Arbeit behandelt diese Auswahl als Beobachtungsbedingung. Ein Hash schützt die gespeicherte Auswahl, nicht vor dem Umstand, dass andere Perspektiven fehlen.

7. Erfassungsprotokoll

Ein Erfassungsprotokoll beginnt mit dem HTTP-Abruf. Die Antwort wird geparkt und auf ein internes Feedformat abgebildet. Anschließend prüft der Prozess, ob die Quellenkennung bereits bekannt ist. Nur unbekannte Meldungen durchlaufen teure Nachladevorgänge. Diese Reihenfolge spart Ressourcen und verhindert, dass ein wiederholter Abruf eines unveränderten Screenshots erneut einen Datensatz produziert.

Die Normalisierung liest Titel, URL, Inhalt, Autor und Zeit aus unterschiedlichen möglichen Feldern. Datumsangaben werden nach UTC gebracht, soweit die Eingabe interpretierbar ist. Fehlende Werte bleiben als fehlende Werte erkennbar. Eine Normalisierung darf nicht stillschweigend aus einem fehlenden Artikeltext einen vollständigen Artikel machen.

Die Verarbeitung jedes Records produziert neben dem Record gegebenenfalls Bildbytes. JSONL speichert relative Bildpfade, SQLite speichert BLOBs zusätzlich. Der Speicherakt aktualisiert die jeweilige Kette. Danach kann ein

Laufzeitresultat die Zahl eingefügter Records, den Endhash und die Shardnummer zurückgeben.

8. Unvollständige Feeds und Nachladen

Die RSS-Feeds enthalten in vielen Fällen nur einen Anreißer. Der Anreißer kann für einen Überblick genügen, repräsentiert aber nicht den gesamten Nachrichtentext. Diese Differenz ist für wissenschaftliche Nutzung entscheidend: Eine spätere Inhaltsanalyse darf nicht behaupten, der kurze Feedtext sei der vollständige Artikel gewesen.

Der TAZ-Codec ruft deshalb den vollständigen Artikel über den Feedlink ab und wandelt strukturierte Artikelabsätze in einen gespeicherten Inhalt um. Der Zusatzabruf ist eine zweite Beobachtung mit eigener Fehlerquelle. Wenn der Feed erreichbar ist, der Artikel aber nicht, muss der Prozess diesen Unterschied im Laufzeitstatus sichtbar machen.

Der Screenshotcodec wählt eine andere Strategie. Er speichert den visuellen Zustand der verlinkten Seite. Dies ist hilfreich, wenn Textstruktur, Bilder, Überschriften oder Seitengestaltung

selbst Forschungsgegenstand sind. Ein Screenshot ist jedoch keine textuelle Vollständigkeitsgarantie. Dynamische Module, Videos und nachgeladene Inhalte können fehlen oder variieren.

9. Codecarchitektur

Codecs kapseln Quellenwissen. Der allgemeine Daemon muss nicht wissen, ob ein Artikel aus JSON, XML, einer TAZ-Seite oder einer Chromium-Aufnahme stammt. Er erwartet ein vorbereitetes Record und eine optionale Menge von Bildbytes. Diese Schnittstelle begrenzt die Zahl der Sonderfälle im Kernsystem.

Versionierte Codecs trennen alte und neue Hashverträge. v0 bleibt für historische Records registriert. v1 ergänzt Metadatenhashes und nimmt sie in die Hashberechnung auf. Damit kann ein gemischter Bestand weiterhin gelesen werden, ohne alte Records mit einem neuen Regelwerk neu zu interpretieren.

Ein Codec ist zugleich eine wissenschaftliche Behauptung über die Quelle. Der Name dokumentiert nicht nur eine

Softwareklasse, sondern eine Verarbeitungsentscheidung. Wer einen SCREENv1-Record untersucht, muss Browserbedingungen und Bannerentfernung berücksichtigen. Wer einen TAZv1-Record untersucht, muss den ergänzenden Artikelabruf berücksichtigen.

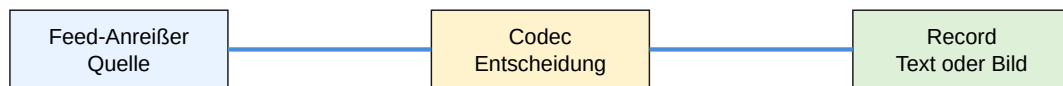


Abbildung 1: Der Codec entscheidet, welche Form der unvollständige Feed annimmt.

10. Datenmodell

Das Recordmodell enthält Identität, Inhalt, Quelle, Zeit, Verarbeitung und Kettenbezug. `source_id` dient der Deduplizierung. `source_url` verweist auf die beobachtete Ressource. `title`, `content` und `author_name` bilden die semantische Darstellung. `published_at` und `retrieved_at` trennen Quellenzeit und Beobachtungszeit. `images` enthält Informationen zu visuellen Ressourcen.

previous_hash und hash bilden die Integritätsschicht. Die v1-Felder schema_version, schema_hash, codec_version, codec_hash, hash_function_version und hash_function_hash bilden die Verarbeitungsprovenienz. Sie machen sichtbar, mit welcher Definitionsdatei und welchem Regelwerk der Record erzeugt wurde.

SQLite übernimmt die Felder in relationaler Form. JSONL bewahrt sie als JSON-Objekt. Alte Tabellen werden bei kompatibler Altstruktur um Metadatenfelder erweitert. Alte Zeilen bleiben leer in diesen Feldern und werden anhand ihres v0-Codecs geprüft. Diese Migration bewahrt historische Records, statt sie in eine neue Version umzuschreiben.

11. Kanonisierung

Eine Hashfunktion sieht keine Bedeutung, sondern Bytes. Deshalb muss festgelegt werden, wie ein Record in Bytes verwandelt wird. News-Hash sortiert JSON-Schlüssel, verwendet UTF-8 und verzichtet auf zusätzliche Trennzeichen.

Die Hashfunktion selbst ist SHA-256, deren Zweck die Erkennung von Nachrichtenänderungen ist [9].

Die Kanonisierung entscheidet, welche Unterschiede relevant sind. Ein anderer Abrufzeitpunkt gehört nicht zum Inhaltsmaterial. Eine andere veröffentlichte Zeit kann dagegen relevant sein. Die Bildmetadaten werden als Teil der Recorddarstellung einbezogen. Eine Änderung des Feldmappings ist eine Änderung des Hashvertrags und benötigt deshalb eine neue Version.

```
H(i) = SHA-256(canonical-json(record(i), previous-hash(i)))  
previous-hash(i+1) = H(i)
```

12. Hashkette

Der erste Record beginnt mit 64 Nullen als Genesiswert. Jeder weitere Record nimmt den Digest des Vorgängers auf. Dadurch entsteht eine Reihenfolge, die nicht nur einzelne Objekte, sondern auch ihre Nachbarschaft prüfbar macht. Wird ein früher Record verändert, weichen seine Hashwerte und die folgenden Vorgängerbezüge ab.

Die Hashkette verhindert keine falsche Eingabe. Ein Betreiber könnte falsche Daten korrekt hashen. Sie verhindert auch nicht den Verlust eines gesamten Archivs. Ihre Aufgabe ist enger: Sie zeigt, ob eine vorliegende Folge mit ihrem dokumentierten Entstehungszustand übereinstimmt.

Die Validierung berechnet jeden Hash neu. Sie prüft zusätzlich, ob `previous_hash` dem erwarteten Vorgänger entspricht. Ein gezielter Shardlauf übernimmt den Endhash des vorherigen Shards als Startpunkt. Ein vollständiger Lauf beginnt beim Genesiswert.

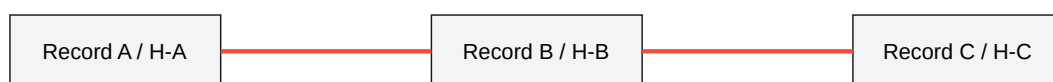


Abbildung 2: Jeder spätere Record trägt die Prüfreferenz seiner Vorgänger.

13. Versionierte Definitionen

Die JSON-Dateien im Data-Verzeichnis sind Teil der Archivumgebung. `schema-v0` und `schema-v1` dokumentieren Recordfelder. `codecs-v0` und `codecs-v1` dokumentieren die Codecfamilie. `hash-functions-v0` und `hash-functions-v1`

dokumentieren Algorithmus, Kanonisierung und Kettenregel. Eine bestehende Definition wird nicht überschrieben, wenn eine neue Fassung benötigt wird.

Beim Erzeugen eines v1-Records werden die SHA-256-Digests dieser Dateien berechnet. Der Record speichert sowohl Version als auch Hash. Ändert jemand eine Definitionsdatei ohne Versionswechsel, kann der spätere Prüfer die Abweichung erkennen. Diese Regel bindet Datenobjekt und technische Entstehungsbedingungen zusammen.

Eine Definition ist keine vollständige formale Spezifikation des Quellcodes. Sie dokumentiert die Eigenschaften, die für die Archivinterpretation relevant sind. Eine zukünftige Forschungsarbeit könnte diese JSON-Formate durch maschinenlesbare Schemas, Signaturen und Abhängigkeiten erweitern.

14. Parallele Speicher

JSONL und SQLite sind zwei Sichtweisen auf denselben Archivbestand. JSONL ist einfach zu exportieren, SQLite ist

schnell zu durchsuchen. Beide werden getrennt gehasht, weil ihr Schreibfortschritt auseinanderfallen kann. Ein Prozessabbruch zwischen den Formaten wird dadurch sichtbar, ohne einen der beiden Zustände heimlich zu verwerfen.

Die SQLite-Tabelle enthält zusätzlich die Anchor-Artefakte. Manifest und OTS-Proof werden als BLOB mit Datum, Typ und Dateiname gespeichert. Die externe Datei bleibt für Download und Synchronisierung erhalten. Die doppelte Ablage erhöht die Chance, einen Nachweis zusammen mit dem lokalen Bestand wiederzufinden.

Shards begrenzen die Dateigröße. Die logische Kette läuft über Shardgrenzen hinweg. Ein Manifest nennt die Nummer des Shards, der seinen Endhash enthält. Eine Diagnose kann dadurch gezielt eine Datei prüfen, statt den gesamten Bestand zu laden.

15. Fehlermodelle

Ein Feed kann ausfallen, eine Quelle kann eine leere Antwort liefern oder ein Artikel kann nicht nachgeladen werden. Der

Daemon isoliert Fehler nach Quelle. Fehlerzähler und Laufzeitlog sind flüchtige Betriebsinformationen. Der unveränderliche Recordbestand wird von diesen Zuständen getrennt.

Ein Prozessabbruch kann einen Speicher vorauslaufen lassen. Die getrennten Endhashes machen diesen Zustand sichtbar. Beim Wiederanlauf werden bekannte IDs und letzte Hashes pro Format ermittelt. Eine automatische Reparatur wird vermieden, weil sie historische Zustände verdecken könnte.

Eine beschädigte Datei wird von der Validierung als konkreter Fehler mit Format, Shard und Recordnummer gemeldet. Dieser Detailgrad erleichtert die Sicherung des betroffenen Abschnitts und eine spätere Wiederherstellung. Ein Fehlerstatus ist kein Beweis, dass der gesamte Bestand verloren ist.

16. Zeitdimension

News-Hash unterscheidet Veröffentlichungszeit, Abrufzeit, Manifestdatum und Proofzeitpunkt. Diese vier Werte werden

nicht vertauscht. Die Veröffentlichung stammt aus der Quelle. Der Abruf dokumentiert die Beobachtung. Das Manifest fasst einen Tageszustand zusammen. Der Proof begrenzt, wann dieser Zustand spätestens existierte.

Die Browseroberfläche kann UTC-Werte lokal anzeigen. Wissenschaftliche Auswertungen sollten dennoch den kanonischen UTC-Wert verwenden. Eine lokale Anzeige ist eine Präsentation, kein neuer Archivwert.

Ein späteres Manifest kann einen über mehrere Tage gewachsenen Kettenzustand beschreiben. Seine Shardnummer und sein Endhash machen den Bezug konkret. Ein Proof ohne lokalen Recordbestand beweist nur die Existenz einer Bytefolge, nicht die Geschichte jedes enthaltenen Records.

17. Manifestbildung

Das Manifest enthält Quelle, Speichername, Datum, beide Endhashes und die beiden letzten Shardnummern. Es enthält keine Nachrichtentexte. Der kleine Umfang ermöglicht

öffentliche Veröffentlichung und unabhängige Zeitverankerung.

Beim Stamping wird eine Textdatei an OpenTimestamps übergeben. Die OTS-Datei wird nach erfolgreichem Prozess in SQLite abgelegt und gemeinsam mit den Backups verwaltet. Die tägliche Datei wird bei einem neuen Lauf für denselben Tag nicht beliebig vervielfacht.

Manifestprüfung sucht den benannten Shard und vergleicht dessen letzten Hash. Legacy-Manifeste ohne Shardfelder können den passenden Shard über den Hash finden. Neue Manifeste sind dennoch vorzuziehen, weil ihre Zuordnung explizit und schneller ist.

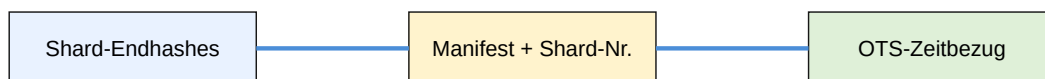


Abbildung 3: Lokale Ketten, Manifest und externer Zeitbezug in drei Prüfstufen.

18. Backups

Beim Anchoring werden SQLite-Shards in data/sqlite-backups kopiert. Die Backups verwenden dieselben Dateinamen und werden überschrieben. Das ist ein aktueller Sicherungsstand und keine historische Sicherungsreihe.

Ein Backup wird erst nach Speicherung von Manifest und Proof in SQLite erzeugt. Dadurch enthält die Kopie auch die Anchor-Artefakte. Die Records werden nicht neu berechnet, weil das Backup eine Kopie und keine neue Beobachtung ist.

Für dauerhafte Aufbewahrung müssen die Backups selbst weiter versioniert oder auf unveränderliche Medien kopiert werden. Die lokale Überschreibung reduziert den laufenden Speicherbedarf, erhöht aber die Bedeutung externer Sicherungen.

19. Externer Zeitnachweis

OpenTimestamps beschreibt einen Zeitstempel als Nachweis, dass Daten vor einem Zeitpunkt existierten [10]. News-Hash übergibt dafür nur die Manifestdatei. Der Dienst erhält keine Meldung und kein Bild.

Die Prüfung verläuft in drei Schritten: Records validieren, Endhashes mit Manifest vergleichen, Proof prüfen. Ein gültiger Proof mit falschem lokalem Manifestbezug ist kein gültiger Nachweis des Archivs.

GitHub-Synchronisierung und OpenTimestamps haben unterschiedliche Rollen. GitHub bewahrt Dateien auffindbar und versioniert. OpenTimestamps liefert die zeitliche Attestation. Zusammen ergeben sie eine stärkere Veröffentlichung als jeweils allein.

20. Screenshots

Ein Screenshot ist eine visuelle Beobachtung unter Browserbedingungen. Viewport, Fonts, Ladezeiten und dynamische Inhalte beeinflussen das Ergebnis. Der Codec entfernt bekannte Consent- und Cookie-Banner, damit ein Overlay nicht den Archivgegenstand verdeckt.

Die Entfernung ist selektorbasiert und kann nie vollständig allgemeingültig sein. Neue Anbieter benötigen eine Prüfung. Bei n-tv wurde ein dynamischer containerbezogener Consent-

Selektor ergänzt. Der erzeugte Screenshot wird anschließend visuell geprüft.

Die Bilddatei und ihr Hash werden als Teil des Records behandelt. Eine spätere Analyse muss erkennen können, dass ein Bild und kein vollständiger Text gespeichert wurde. Diese Unterscheidung verhindert eine falsche Gleichsetzung von visueller und semantischer Vollständigkeit.

21. Benutzeroberfläche

Das Dashboard verbindet Quellen, Meldungen, Bilder, Laufzeit und Anchorstatus. Es ist eine Zugangsschicht zu den Daten, nicht deren alleinige Beweisinstanz. Lange Hashwerte müssen kopierbar und unterscheidbar angezeigt werden.

Die Detailseite zeigt Record und Vorgängerhash. Eine zukünftige Erweiterung könnte Codec- und Metadatenhashes ausgeben und eine direkte gezielte Validierung anbieten. Ein Status sollte immer beschreiben, welche Ebene geprüft wurde.

Die deutsche Oberfläche bleibt stabil, während Erklärungstexte auf Deutsch und Englisch gepflegt werden.

Technische Schlüssel bleiben sprachunabhängig. Das verhindert, dass Übersetzungen die maschinenlesbare Bedeutung verändern.

22. Monitoring

Ein Heartbeat wird nur gesendet, wenn mindestens eine neue Meldung gespeichert wurde. Ein laufender Daemon ohne neue Meldung erzeugt keinen Erfolgsping. Diese Semantik unterscheidet Prozesslauf von Quellenfortschritt.

Prometheus-Metriken ergänzen den Heartbeat. Fehler, Recordzahlen und Bilder können überwacht werden. Ein fehlender Ping kann dann zusammen mit Quellenfehlern und letzter erfolgreicher Verarbeitung interpretiert werden.

Für selten aktualisierte Quellen sind individuelle Intervalle wichtig. Ein täglicher Feed darf nicht nach wenigen Minuten als fehlerhaft gelten. Monitoringparameter sind daher Bestandteil der Konfiguration.

23. Datenschutz

Nachrichtentexte und Bilder können personenbezogene oder urheberrechtlich geschützte Informationen enthalten. Lokaler Betrieb reduziert externe Übertragung, beseitigt aber nicht die Verantwortung des Betreibers. Aufbewahrung und Zugriff müssen separat bewertet werden.

OpenTimestamps erhält nur Hashmanifeste. Hashes können bei bekannten kleinen Dateien dennoch vergleichbare Informationen liefern. Die Veröffentlichung sollte deshalb bewusst und zweckgebunden erfolgen.

Eine append-only Historie kann mit späteren Löschinteressen kollidieren. Verschlüsselung, Zugriffsbegrenzung und dokumentierte Verwaltungsvorgänge können Risiken reduzieren, ersetzen aber keine rechtliche Entscheidung.

24. Bedrohungsmodell

Das Modell betrachtet nachträgliche Recordänderungen, Shardverlust, manipulierte Manifeste, unvollständige Abrufe und falsche Eingaben. Die Hashkette erkennt viele

nachträgliche Veränderungen, sofern eine vertrauenswürdige Vergleichsbasis vorhanden ist.

Sie schützt nicht gegen einen Betreiber, der von Anfang an falsche Inhalte speichert. Sie schützt nicht gegen vollständiges Löschen ohne unabhängige Kopie. Sie beweist nicht, dass ein nicht abgerufener Artikel nie existiert hat.

Die WebUI ohne Authentifizierung ist nur für geschützte Netze geeignet. Abruf- und Shutdownendpunkte dürfen nicht öffentlich exponiert werden. Credentials benötigen sichere Dateirechte.

25. Qualitätssicherung

Tests prüfen Settings, Feeds, Codecs, Ketten, Shards, SQLite, Anchoring, WebUI und Validierung. Die Testebenen haben unterschiedliche Reichweiten. Ein Unit-Test beweist kein stabiles Verhalten einer externen Website.

Browserprüfungen verwenden domcontentloaded und eine Renderwartezeit. Netzwerkverbindungen können dauerhaft

bleiben. Ein Screenshottest sollte daher nicht auf networkidle warten.

Dokumentation wird als Qualitätsartefakt behandelt. Architektur, Vision, Anforderungen und Anwenderhilfe müssen denselben technischen Zustand beschreiben. Widersprüche erschweren eine spätere Prüfung.

26. Reproduzierbarkeit

Reproduzierbarkeit hängt von Quell-URL, Konfiguration, Codec, Softwareversion, Browser, Viewport und Zeitpunkt ab. JSON-Feeds sind oft strukturell stabiler als dynamische Seiten. Ein Screenshot ist stärker an eine konkrete Umgebung gebunden.

Die Lockdatei und Metadatenhashes dokumentieren den Software- und Regelstand. Konfigurationssnapshots und Rohantworten wären zusätzliche sinnvolle Forschungsartefakte. Ihr Datenschutz- und Speicheraufwand muss berücksichtigt werden.

Eine unabhängige Person benötigt Recordbestand, Definitionen, Manifest und Proof. Erst ihr Zusammenspiel macht die technische Aussage reproduzierbar.

27. Evaluation

Die Evaluation prüft Normalbetrieb, Wiederholungsabruf, Fehlerfeed, beschädigten Record, Sharda Auswahl, Legacy-Manifest, Anchor und Backup. Jedes Szenario besitzt eine erwartete Ausgabe und eine definierte Grenze.

Die vorhandenen Tests zeigen, dass neue Records mit v1-Metadaten erzeugt werden, alte Records validierbar bleiben und Manifestartefakte überschrieben in SQLite gespeichert werden. Die Tests ersetzen keine langfristige Betriebsbeobachtung.

Ein Forschungsexperiment könnte über mehrere Monate Latenz, Quellenabdeckung, Fehler, Screenshotänderungen und Backupdauer messen. Der Prototyp stellt dafür die notwendigen Messpunkte bereit.

28. Datenbankmigration

Die SQLite-Migration ergänzt alte Records um neue Metadatenfelder, ohne die historischen Hashes neu zu berechnen. Fremde oder unklare Schemas werden als Legacy gesichert. Diese Vorsicht verhindert Datenverlust durch automatische Annahmen.

Die Anchor-Artefakttabelle ist unabhängig von Records. Das erlaubt das Speichern von Manifest und Proof, ohne Nachrichtenhashes zu verändern. Ein Prüfer kann beide Tabellen separat untersuchen.

JSONL benötigt keine Schemaänderung auf Dateiebene. Neue Felder werden in neuen Records ergänzt. Alte Zeilen bleiben als v0-Records erkennbar.

29. Shards und Skalierung

Shards begrenzen Dateigrößen und erlauben inkrementelle Kopien. Die logische Kette läuft über die physischen Grenzen. Manifeste nennen den letzten Daten-Shard mit Endhash.

Die gezielte Validierung reduziert Prüfkosten. Ein gesamter Lauf liest alle Records, ein Shardlauf prüft einen Abschnitt

relativ zum Vorgängerhash. Diese Modi sollten im Prüfprotokoll unterschieden werden.

Bei großen Bildbeständen werden Bildbytes nicht für jede Kettenprüfung erneut gelesen. Der Record enthält bereits den Bildhash. Eine separate Bildintegritätsprüfung kann als zukünftiges Werkzeug ergänzt werden.

30. Quellenvergleich

Mehrere Quellen können dasselbe Ereignis unterschiedlich beschreiben. News-Hash bewahrt die Eingaben, erzeugt aber keine automatische Ereignisidentität. Ein späterer Vergleich muss Titel, Zeit, URL und Inhalt semantisch untersuchen.

Quellenabdeckung bleibt eine Auswahlentscheidung. Ein Feedarchiv ist keine vollständige Repräsentation der Gesellschaft. Diese Grenze muss in Analysen ausgewiesen werden.

Gerade die technische Gleichbehandlung mehrerer Quellen kann Unterschiede sichtbar machen, ohne sie zu bewerten. Das

ist ein möglicher Ausgangspunkt für Medienvergleich und Quellenkritik.

31. Nutzerverständnis

Hashwerte sind für viele Nutzer abstrakt. Die Oberfläche muss erklären, dass ein Hash eine technische Identifikation und kein Gütesiegel ist. Ein Anchor zeigt Existenz, nicht Wahrheit.

Ein Hinweis auf den geprüften Shard kann Vertrauen erhöhen, wenn er mit einer verständlichen Erklärung verbunden ist. Ohne Erklärung kann eine zusätzliche Nummer auch Verwirrung erzeugen.

Benutzerforschung sollte testen, ob Personen Manifest, Record, Screenshot und Proof richtig auseinanderhalten. Verständlichkeit ist eine funktionale Eigenschaft eines Prüfwerkzeugs.

32. Governance

Ein lokales Archiv benötigt Regeln für Quellenaufnahme, Codecänderung, Versionsnummern, Backups und

Zugriffsrechte. Diese Regeln sollten öffentlich dokumentiert werden, wenn das Archiv wissenschaftlich genutzt wird.

Änderungen am Hashvertrag erfordern eine neue Version. Eine Änderung ohne neue Version beschädigt die Interpretierbarkeit alter Records. Code-Review und Testnachweise sind Teil dieser Governance.

Die Entscheidung, welche Quellen dauerhaft erfasst werden, sollte nicht nur technisch, sondern auch redaktionell und rechtlich begründet werden.

33. KI-gestützte Entwicklung

Die fast vollständige KI-Erzeugung von App und Arbeit ist selbst eine Provenienzfrage. Generative Systeme erzeugen plausible Strukturen, können aber unbelegte Details oder falsche Zusammenhänge liefern. Jede Aussage benötigt deshalb eine menschlich gesetzte Prüfregel.

Der transparente Hinweis beschreibt keine Minderwertigkeit. Er macht die Produktionsweise nachvollziehbar und

ermöglicht eine angemessene Prüfung. Quellcode, Tests und externe Quellen bleiben entscheidend.

Das Motto verbindet Werkzeug und Zweck: „Mit KI gebaut, mit Liebe verfeinert, für neugierige Menschen.“ Die menschliche Verfeinerung besteht in Auswahl, Korrektur und Verantwortung.

34. Rechtliche Abwägung

Archivierung kann mit Datenschutz, Persönlichkeitsrechten und Urheberrecht kollidieren. Ein technisch unveränderlicher Record darf nicht als automatische Erlaubnis zur dauerhaften Speicherung verstanden werden.

Der lokale Betrieb kann Zugriff begrenzen, ersetzt aber keine Rechtsprüfung. Quellenwahl und Aufbewahrungsdauer gehören in eine verantwortliche Betriebsordnung.

Ein Manifest ohne Nachrichtentext reduziert den externen Abfluss, hebt lokale Pflichten aber nicht auf.

35. Vergleich mit Vollarchiven

Vollarchive zielen auf maximale Ressourcenerhaltung. News-Hash zielt auf einen prüfbaren lokalen Ausschnitt. Die Verfahren sind nicht konkurrierend, sondern können sich ergänzen.

Ein Vollarchiv kann historische Darstellung liefern, während News-Hash die Integrität einer ausgewählten Kopie sichtbar macht. Die Auswahl- und Lückeninformationen müssen dabei gemeinsam betrachtet werden.

Die technische Einfachheit eines lokalen Dienstes kann für kleine Forschungsvorhaben ein Vorteil sein, wenn seine Grenzen transparent bleiben.

36. Verteilte Perspektiven

Mehrere unabhängige Archive könnten Tagesmanifeste gegenseitig bestätigen. Dadurch würde ein einzelner Betreiber weniger wichtig. Gleichzeitig entstünden neue Regeln für Teilnehmer, Konflikte und Datenschutz.

Eine solche Erweiterung wäre keine notwendige Voraussetzung für die lokale Kette. Sie wäre ein

eigenständiges Forschungsfeld mit eigenen Vertrauensannahmen.

Die jetzige Architektur bietet eine kleine, kontrollierte Basis, auf der solche Modelle experimentell untersucht werden könnten.

37. Langzeitaufbewahrung

Langzeitaufbewahrung verlangt mehr als aktuelle Lesbarkeit. Abhängigkeiten, Browser, Dateiformate und Definitionen müssen auffindbar bleiben. Versionierte JSON-Metadaten sind ein erster Baustein.

Die Aufbewahrung von JSONL, SQLite, Bildern, Manifests, Proofs und Konfiguration bildet ein Paket. Fehlt ein Teil, kann die spätere Prüfung eingeschränkt sein.

Regelmäßige Prüfungen und dokumentierte Migrationen sind für ein dauerhaftes Archiv wichtiger als die bloße Existenz alter Dateien.

38. Forschungsdatenmanagement

Ein Forschungsprojekt sollte Rohdaten, transformierte Records, Konfiguration, Softwareversion und Prüfprotokoll unterscheiden. News-Hash kann diese Ebenen in Verzeichnisstruktur und Metadaten abbilden.

Die Veröffentlichung von Daten muss Datenschutz und Rechte beachten. Ein wissenschaftlicher Datensatz ist nicht automatisch ein öffentlicher Volltextbestand.

Eine dokumentierte Auswahlstrategie ermöglicht späteren Nutzern, Ergebnisse angemessen einzuordnen und nicht mit einer vollständigen Nachrichtengrundgesamtheit zu verwechseln.

39. Architekturkritik

Die lokale Kopplung von Daemon und WebUI vereinfacht Deployment, begrenzt aber horizontale Skalierung. Die parallelen Speicher erhöhen Robustheit, erhöhen aber Wartungsaufwand. Die Codecmodularität erweitert Abdeckung, erzeugt aber quellspezifische Unsicherheiten.

Die tägliche Manifestbildung ist ein praktikabler Kompromiss. Feingranulare Proofs wären präziser, aber teurer und datenschutzintensiver.

Die Arbeit bewertet diese Entscheidungen nicht als universell optimal. Sie sind für einen nachvollziehbaren Prototypen plausibel und offen für Messung und Revision.

40. Schlussfolgerung

News-Hash zeigt, wie ein lokaler Importdienst aus flüchtigen Nachrichten eine prüfbare technische Spur erzeugen kann. Die zentrale Leistung liegt nicht in einer neuen Kryptografie, sondern in der Verbindung von Normalisierung, Versionierung, Speicher, Manifest und Prüfung.

Die Arbeit macht sichtbar, dass unvollständige Feeds, nachgeladene Artikel, Screenshots, Shards, Backups, Metadaten und externe Zeitbezüge zusammen betrachtet werden müssen. Nur dann lässt sich präzise sagen, was gespeichert wurde und welche Aussage ein Hash erlaubt.

Die wichtigste Grenze bleibt bestehen: Integrität ist nicht Wahrheit. Ein ehrliches, technisch prüfbares Archiv kann trotzdem einen begrenzten oder fehlerhaften Ausschnitt bewahren. Gerade deshalb gehören Transparenz, Quellenkritik und menschliche Verantwortung zum System.

News-Hash ist somit ein Forschungswerkzeug für neugierige Menschen: „Mit KI gebaut, mit Liebe verfeinert, für neugierige Menschen.“

Literatur

- [1] Consultative Committee for Space Data Systems: *Reference Model for an Open Archival Information System (OAIS)*, CCSDS 650.0-M-2, 2012. <https://public.ccsds.org/Pubs/650x0m2.pdf>.
- [2] Groth, P.; Moreau, L.: *PROV-Overview*, W3C Working Group Note, 2013. <https://www.w3.org/TR/prov-overview/>.
- [3] Van de Sompel, H.; Nelson, M.; Sanderson, R.: *HTTP Framework for Time-Based Access to Resource States - Memento*, RFC 7089, 2013. <https://www.rfc-editor.org/rfc/rfc7089>.
- [4] National Institute of Standards and Technology: *Secure Hash Standard*, FIPS PUB 180-4, 2015. <https://doi.org/10.6028/NIST.FIPS.180-4>.
- [5] OpenTimestamps: *A timestamping proof standard*, <https://opentimestamps.org/>.
- [6] News-Hash-Projekt: *Architektur, Anforderungen und WebUI-Dokumentation*, Projektstand 2026.

Diese vollständig neu formulierte Arbeit beschreibt den Projektstand vom 17. August 2026. Die PDF wurde aus dieser HTML-Quelle mit Chromium im A4-Format erzeugt.